

PROJECT REPORT

TalentBridge

A Placement Management System

Author:	Prashant Raj
Program:	B.Tech, Computer Science & Engineering
University:	VIT Bhopal University
Date:	June 2026

1. Title

TalentBridge – A Placement Management System Backend, developed using Spring Boot and MongoDB to streamline the placement process for students, recruiters, and institutional administrators.

The system was built as a RESTful backend that exposes a complete set of APIs for managing students, companies, jobs, and job applications, supported by JWT-based authentication and a dashboard analytics module. The full API surface was documented using Swagger/OpenAPI and verified end-to-end using Postman.

2. Objective

The placement process at most academic institutions involves a large number of manual, repetitive tasks: tracking which companies are visiting, which jobs they are offering, which students are eligible and have applied, and what the outcome of each application is. TalentBridge was built to digitize and centralize this workflow.

The specific objectives of the project were:

- Provide a single backend system to manage student profiles, company records, job postings, and applications.
- Allow students to be matched against jobs using attributes such as skills and CGPA.
- Track the complete lifecycle of a job application, from “Applied” to “Selected” or “Rejected”.
- Expose secure REST APIs protected with JWT authentication so that the system could later support a front-end client or mobile app.
- Provide dashboard-level analytics (total students, companies, jobs, and applications) for quick administrative review.

3. Outcome

By the end of the project, a fully functional REST API backend was delivered and verified. The outcomes achieved are summarized below:

- A working Spring Boot application backed by MongoDB, exposing 25+ REST endpoints across five functional modules: Authentication, Student, Company, Job, and Application.
- Full CRUD (Create, Read, Update, Delete) functionality for Students, Companies, and Jobs.
- Relationship-aware endpoints, such as retrieving a company along with all its job postings, or a student along with all their applications.
- Search functionality across multiple modules — students searchable by name, skill, or CGPA; jobs searchable by title; companies searchable by name.

- Application status tracking with the ability to query applications by student, by job, or by status (e.g. APPLIED, SELECTED, REJECTED).
- A dashboard statistics endpoint returning aggregate counts (total students, companies, jobs, and applications) for at-a-glance reporting.
- JWT-based authentication securing the API, with user registration and login endpoints.
- Complete interactive API documentation generated automatically through Swagger/OpenAPI 3.1, and functional testing of every endpoint carried out using Postman.

Overall, the project demonstrates a production-style backend architecture and validates that all designed endpoints behave correctly with real, persisted data in MongoDB.

4. Tools and Technologies Used

The following tools and technology versions were used to design, build, test, and document the TalentBridge backend:

Tool / Technology	Version	Purpose
Java	17 (LTS)	Core programming language
Spring Boot	3.x	Application framework for building the REST backend
Spring Data MongoDB	3.x	Repository layer for MongoDB persistence
MongoDB	6.x (NoSQL)	Document-oriented database for storing all entities
Spring Security	6.x	Authentication and endpoint authorization
JWT (JSON Web Token)	jjwt 0.11.x	Stateless token-based authentication
Swagger / OpenAPI	OAS 3.1	Auto-generated, interactive API documentation
Maven	3.9.x	Build automation and dependency management
Postman	Desktop App (latest)	Manual and functional API testing

5. System Modules

The backend is organized into five functional modules, each exposed through its own set of REST endpoints:

5.1 Authentication Module

Handles user registration and login, issuing a JWT token on successful authentication. This token is then required to access the protected endpoints of the system.

5.2 Student Module

Provides full CRUD operations on student records (name, email, CGPA, and skills), along with search endpoints by name, skill, and CGPA, and an endpoint to fetch a student together with all their submitted applications.

5.3 Company Module

Provides full CRUD operations on company records (name, location, industry), along with a search-by-name endpoint and an endpoint to fetch a company together with all jobs it has posted.

5.4 Job Module

Provides full CRUD operations on job postings (title, description, salary, linked company), a search-by-title endpoint, and an endpoint to fetch a job together with all applications received for it.

5.5 Application Module

Tracks the relationship between a student and a job, including the current status of the application (APPLIED, SELECTED, REJECTED, and so on). Applications can be created, listed, filtered by student, by job, by status, updated, and deleted.

5.6 Dashboard Module

Exposes a single statistics endpoint that aggregates and returns the total number of students, companies, jobs, and applications currently stored in the system, giving administrators a quick overview of platform activity.

6. Output Screenshots and Explanation

This section presents a curated set of screenshots that demonstrate the working system, captured from the Swagger UI documentation and from Postman API testing.

6.1 Swagger API Documentation – Overview

The screenshot below shows the auto-generated Swagger UI for the TalentBridge API. It lists all six API groups — Dashboard, Authentication, Company, Job, Student, and Application — confirming that the

OpenAPI specification was correctly generated from the Spring Boot controllers and is being served at `/v3/api-docs`.

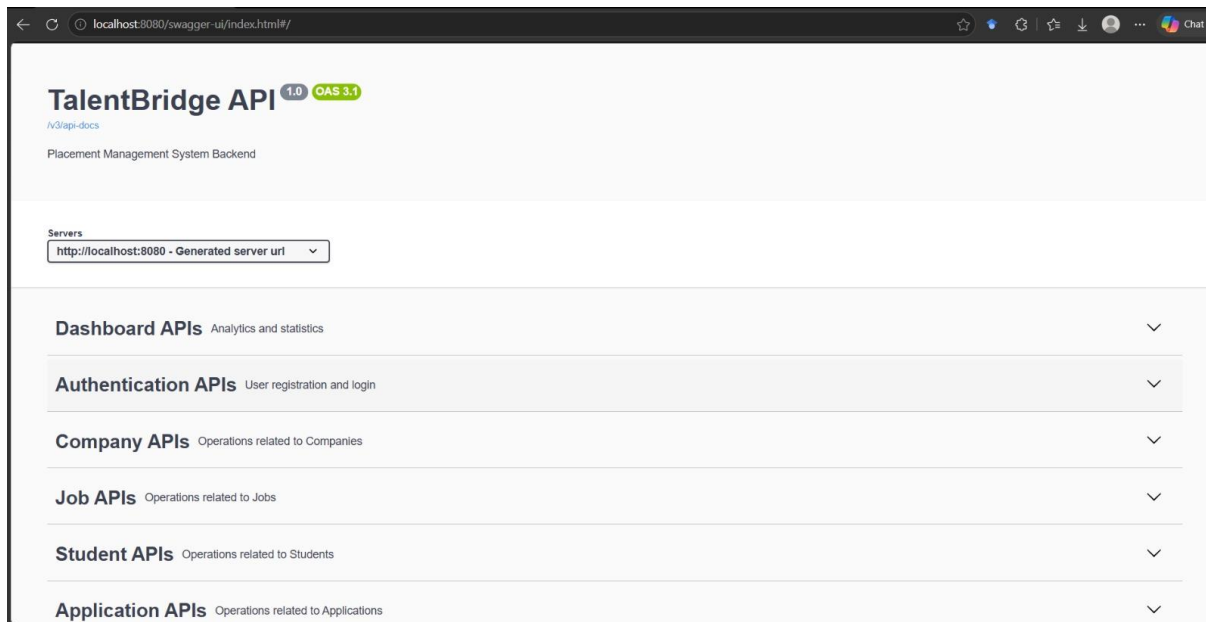


Figure 1: Swagger UI landing page showing all available API groups.

6.2 Student APIs

The Student API group exposes nine endpoints covering full CRUD operations plus three dedicated search endpoints (by skill, by name, and by CGPA) and a relationship endpoint to fetch a student along with their applications. This reflects the requirement that students be discoverable by multiple criteria for job matching.

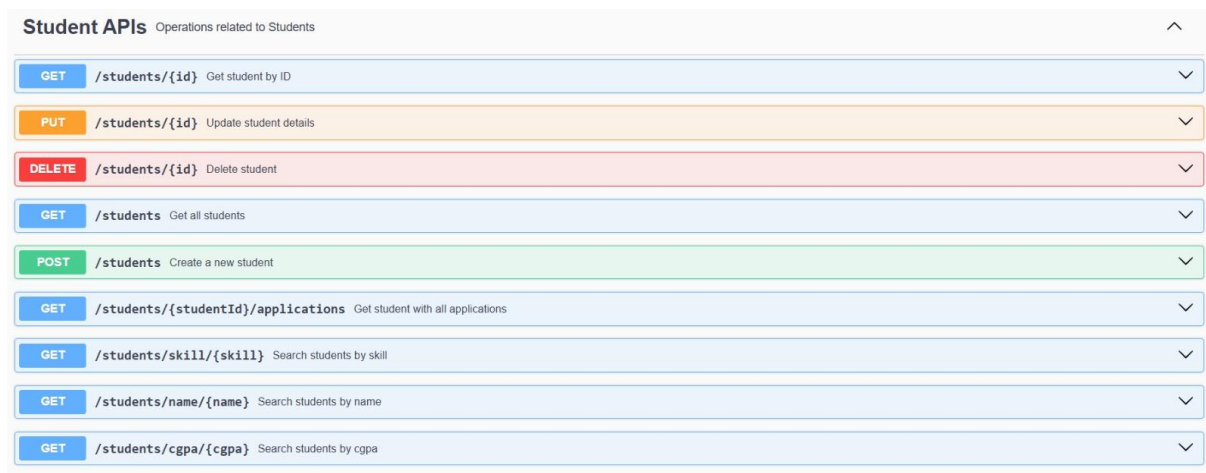
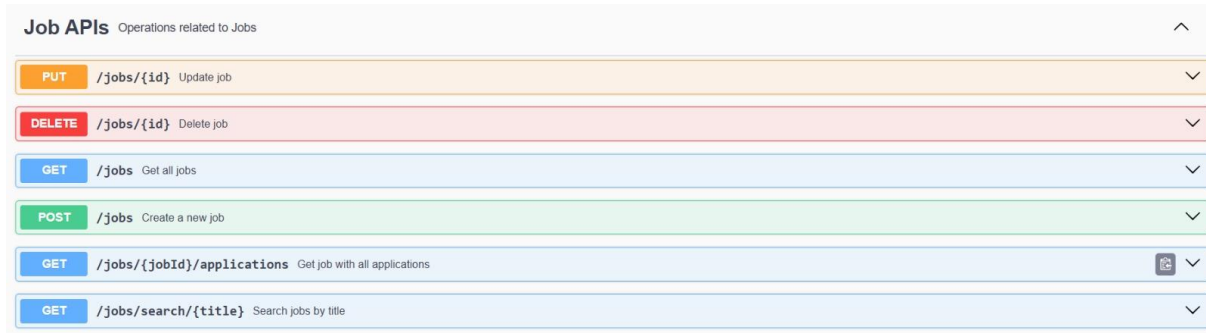


Figure 2: Student API endpoints as listed in Swagger UI.

6.3 Job APIs

The Job API group exposes endpoints to create, update, delete, and list job postings, search jobs by title, and retrieve a job together with all applications submitted against it. Each job record stores its title, description, salary, and the ID of the company that posted it.

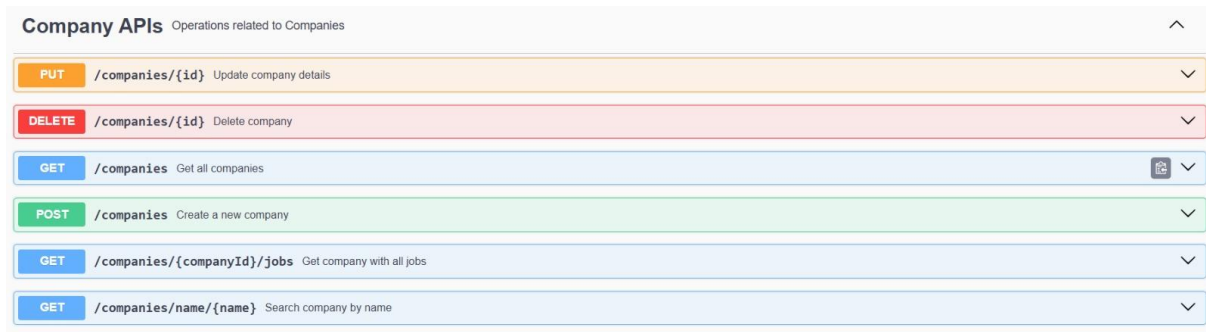
A screenshot of the Swagger UI for the Job APIs. The title is "Job APIs" with a subtitle "Operations related to Jobs". It lists seven endpoints: 1. PUT /jobs/{id} Update job (orange bar). 2. DELETE /jobs/{id} Delete job (red bar). 3. GET /jobs Get all jobs (blue bar). 4. POST /jobs Create a new job (green bar). 5. GET /jobs/{jobId}/applications Get job with all applications (blue bar, includes a schema icon). 6. GET /jobs/search/{title} Search jobs by title (blue bar). Each endpoint has a dropdown arrow on the right.

Job APIs		Operations related to Jobs
PUT	/jobs/{id}	Update job
DELETE	/jobs/{id}	Delete job
GET	/jobs	Get all jobs
POST	/jobs	Create a new job
GET	/jobs/{jobId}/applications	Get job with all applications
GET	/jobs/search/{title}	Search jobs by title

Figure 3: Job API endpoints as listed in Swagger UI.

6.4 Company APIs

The Company API group manages company records and exposes an endpoint to fetch a company along with all jobs it has posted, as well as a search-by-name endpoint, supporting the recruiter side of the placement workflow.

A screenshot of the Swagger UI for the Company APIs. The title is "Company APIs" with a subtitle "Operations related to Companies". It lists six endpoints: 1. PUT /companies/{id} Update company details (orange bar). 2. DELETE /companies/{id} Delete company (red bar). 3. GET /companies Get all companies (blue bar, includes a schema icon). 4. POST /companies Create a new company (green bar). 5. GET /companies/{companyId}/jobs Get company with all jobs (blue bar). 6. GET /companies/name/{name} Search company by name (blue bar). Each endpoint has a dropdown arrow on the right.

Company APIs		Operations related to Companies
PUT	/companies/{id}	Update company details
DELETE	/companies/{id}	Delete company
GET	/companies	Get all companies
POST	/companies	Create a new company
GET	/companies/{companyId}/jobs	Get company with all jobs
GET	/companies/name/{name}	Search company by name

Figure 4: Company API endpoints as listed in Swagger UI.

6.5 Filtering Applications by Status

The screenshot below shows a Postman test of the GET /applications/status/SELECTED endpoint. The response correctly returns only those application records whose status field equals “SELECTED”, each including the student ID, job ID, and application ID — confirming that the status-filtering logic works as intended against live MongoDB data.

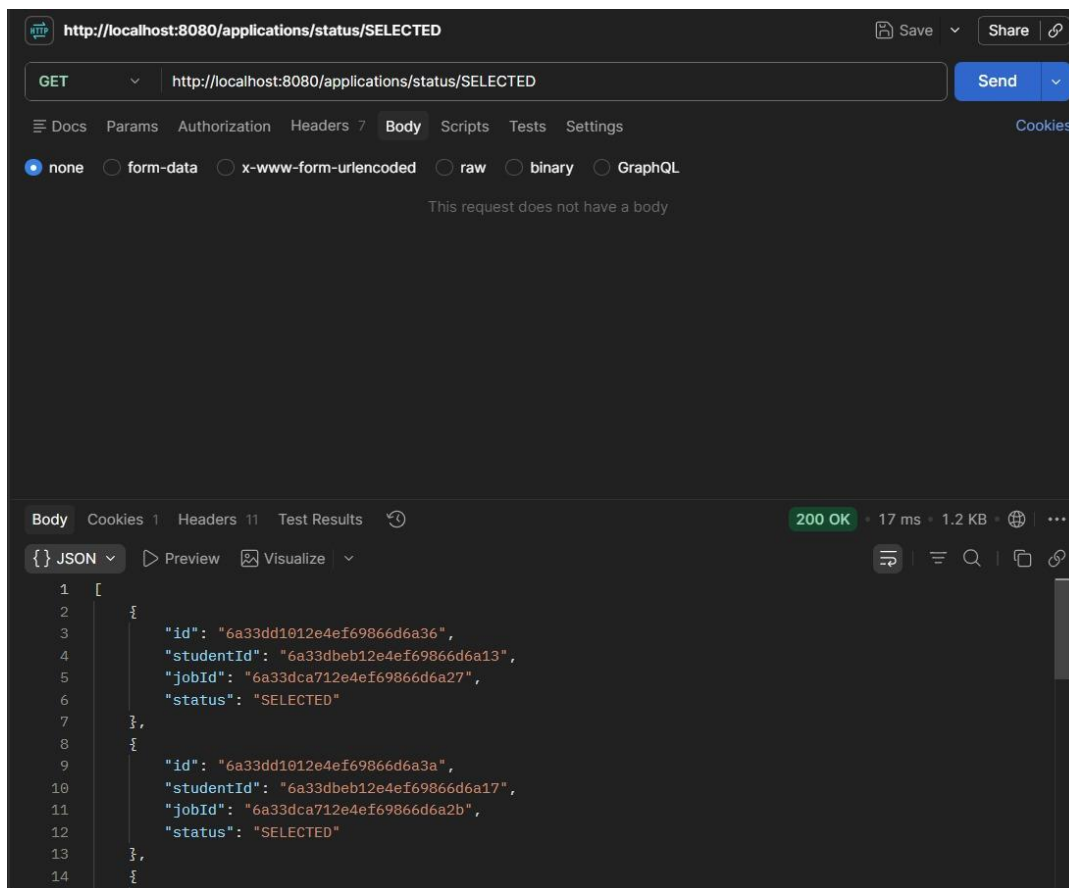


Figure 5: Postman test — retrieving applications filtered by status “SELECTED”.

6.6 Dashboard Statistics

The final screenshot shows the GET `/dashboard/stats` endpoint returning aggregate counts across the system: 15 total jobs, 10 total students, 10 total companies, and 20 total applications. This single endpoint validates that data from all four core collections is being correctly counted and surfaced for administrative reporting.

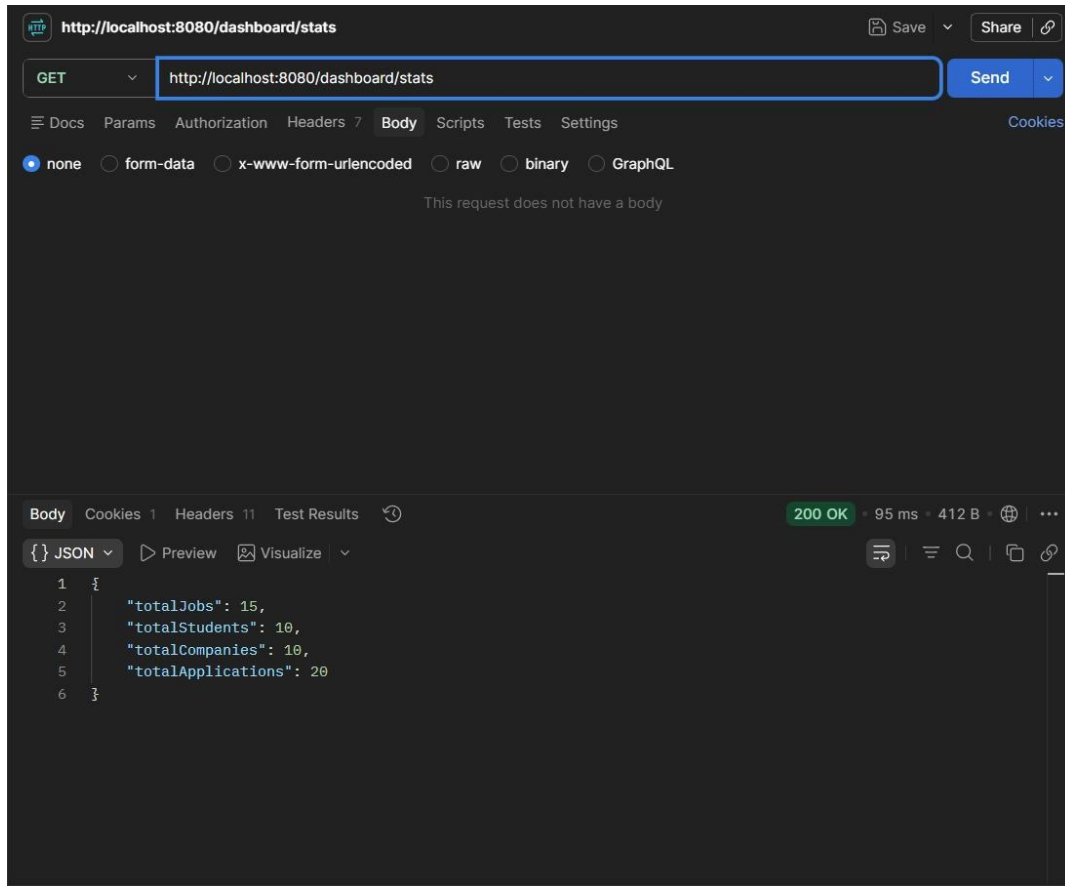


Figure 6: Postman test — dashboard statistics endpoint response.

7. Summary

TalentBridge is a backend system for managing the end-to-end placement process within an academic institution. It was implemented using Spring Boot for the application layer and MongoDB as the persistence layer, with Spring Security and JWT providing stateless authentication for protected endpoints.

The system models four core entities — Student, Company, Job, and Application — and exposes a comprehensive REST API for each, including standard CRUD operations, relationship-aware lookups (for example, a company together with its jobs, or a student together with their applications), and targeted search endpoints (by skill, name, CGPA, title, or status). A dedicated dashboard endpoint aggregates key metrics for quick reporting.

All API behaviour was documented automatically via Swagger/OpenAPI 3.1 and independently verified using Postman, with each endpoint tested against live data stored in MongoDB. The screenshots included in this report confirm that the documented endpoints and their actual runtime responses are consistent with the intended design.

8. Conclusion

The TalentBridge project successfully demonstrates the design and implementation of a multi-module REST API backend using modern Java technologies. By combining Spring Boot, Spring Data MongoDB, and Spring Security with JWT, the project achieves a secure, modular, and testable system architecture suitable for real-world placement management.

The use of Swagger/OpenAPI for documentation and Postman for testing ensured that every endpoint could be independently verified, reducing the risk of integration issues if the backend is later connected to a front-end or mobile client. The dashboard analytics module further adds practical value by giving institutional staff a quick, data-driven view of placement activity.

As a possible direction for future work, the system could be extended with role-based access control (separating student, recruiter, and administrator permissions), email notifications on application status changes, and a front-end interface to make the platform usable directly by end users rather than only through API calls.